

Baseline Chart Explained

The July 2026 Baseline Chart — Explained From Scratch

Purpose of this document. You built a chart that compares every month against a fixed July 2026 baseline. This document explains *every* line of code behind it, in plain language, assuming no prior knowledge of Polymer or Nuxeo. By the end you should be able to explain it confidently to your lead engineer.

PART 0 — The mental model (read this first)

Before any code, understand the shape of the thing.

A **Polymer component** is a custom HTML tag you invent yourself. Ours is `<fid-analytical-dashboard>`. Inside it there are three zones:

Zone	What lives there	In our file
Template	The HTML you see on screen	<code><template>...</template></code>
Properties	The component's memory (its variables)	<code>properties: { ... }</code>
Methods	The component's functions	<code>_baselineValues: function() {...}</code>

The magic that ties them together is **data binding**. You write a property's name inside `[[ ]]` in the HTML, and Polymer keeps that spot on screen in sync with the property's value. Change the property in JavaScript → the screen updates by itself. You never write "find this element and update it". That is the whole idea.

The one-sentence summary of our feature:

Two data feeds fetch per-month numbers starting from a *fixed* July 2026 date; helper functions reshape that raw data into chart bars and into a percentage table; Polymer bindings paint both onto the screen automatically.

PART 1 — The queries (where the numbers come from)

The code

```
<!-- ===== SLOT 6: JULY 2026 BASELINE =====
  These two feeds are deliberately anchored to a FIXED start date
  (Jul 2026) rather than the period selector, so the baseline chart
  always begins at July 2026 and runs to the chosen end date. -->
<nuxeo-repository-data
  start-date="[[baselineStart]]"
  end-date="[[_extendEndDate(endDate)]]"
  with-date-intervals="month"
  date-field="dc:created"
  data="{{baselineDocsPerMonth}}"
  index="[[index]]">
</nuxeo-repository-data>

<nuxeo-repository-data
  start-date="[[baselineStart]]"
  end-date="[[_extendEndDate(endDate)]]"
  with-date-intervals="month"
  date-field="dc:created"
  metrics="sum(file:content.length)"
  data="{{baselineStoragePerMonth}}"
  index="[[index]]">
</nuxeo-repository-data>
```

What `<nuxeo-repository-data>` actually is

It is **not** a visible element. It draws nothing. It is a Nuxeo-provided component whose only job is: "*go ask Elasticsearch a question and put the answer in a property.*" Think of it as a query engine wearing an HTML costume.

Line by line

`start-date="[[baselineStart]]"` The date the query starts counting from. We bind it to our own property `baselineStart`, which is hard-coded to `'2026-07-01'`. **This single line is the heart of the whole feature.** The Overall chart binds `start-date` to `startDate` (which the period dropdown changes). We deliberately bind to `baselineStart` instead, so *no matter what the user picks in the dropdown, this query always starts at July 2026.*

`end-date="[[_extendEndDate(endDate)]]"` Where the query stops. This one **does** follow the user's selection (`endDate`). `_extendEndDate()` is a helper that pushes the date to 23:59:59 of that day, so documents created *today* are included rather than cut off at midnight.

Polymer concept — a computed binding. `[[_extendEndDate(endDate)]]` means "call the method `_extendEndDate`, pass it `endDate`, and use whatever it returns." Polymer re-runs it automatically whenever `endDate` changes.

`with-date-intervals="month"` Tells Elasticsearch: don't give me one big total, **bucket the results by month**. This is what turns a single number into a series like Jul, Aug, Sep...

`date-field="dc:created"` Which date to bucket on. `dc:created` is the Nuxeo field for "when the document was created". (`dc` = Dublin Core, a standard metadata schema.)

`metrics="sum(file:content.length)"` (*second feed only*) By default the query **counts documents**. Adding this metrics line changes the question to: "sum up the byte-size of every file instead." That is why:

- Feed 1 (no metrics) → **document counts** → `baselineDocsPerMonth`
- Feed 2 (`metrics=sum(...)`) → **storage in bytes** → `baselineStoragePerMonth`

Two feeds, same date window, two different measurements.

`data="{{baselineDocsPerMonth}}"` Where to put the answer.

Polymer concept — `{{ }}` vs `[[ ]]`. This is the most important syntax distinction in Polymer:

- `[[foo]]` = **one-way, downward**. Parent → child. "Here is a value for you."
- `{{foo}}` = **two-way**. The child can write *back* into the parent's property.

We use `{{ }}` here because `nuxeo-repository-data` needs to **push the query result up into our component's property**. That is data flowing *upward*, so it must be `{{ }}`. Everywhere we merely *display* a value we use `[[ ]]`.

`index="[[index]]"` Which Elasticsearch index to query. Passed straight through.

What comes back

An array of month buckets:

```
baselineDocsPerMonth = [
  { key: '2026-07', value: 1000 }, // July : 1000 documents
  { key: '2026-08', value: 1500 }, // August: 1500 documents
  { key: '2026-09', value: 750 }
]

baselineStoragePerMonth = [
  { key: '2026-07', value: 524288000 }, // July : bytes
  { key: '2026-08', value: 786432000 },
  { key: '2026-09', value: 419430400 }
]
```

Each entry has a **key** (the month) and a **value** (the measurement). Remember this shape — every helper below consumes it.

PART 2 — The properties (the component's memory)

// ---- Slot 6: July 2026 baseline ----

```

baselineStart:      { type: String, value: '2026-07-01' },
baselineDocsPerMonth: { type: Array, value: function() { return []; } },
baselineStoragePerMonth: { type: Array, value: function() { return []; } },

_baselineGrowth: {
  type: Array,
  value: function() { return []; },
  computed: '_computeBaselineGrowth(baselineStoragePerMonth, baselineDocsPerMonth)'
},

```

baselineStart: { type: String, value: '2026-07-01' } A plain string. value: is its starting value. **Change this one line and the whole feature re-baselines to a different month.** That is the only place the date lives.

baselineDocsPerMonth: { type: Array, value: function() { return []; } }

Two things to notice:

1. **type: Array** — Polymer needs to know the type so it can watch it correctly.
2. **value: function() { return []; }** — the default is written as a *function*, not just []. **This is mandatory in Polymer for objects and arrays.** If you wrote value: [], every instance of the component would share the *same* array in memory (a classic JavaScript bug). By returning a fresh [] from a function, each instance gets its own. Non-negotiable rule.

The computed property — the clever bit

```

_baselineGrowth: {
  computed: '_computeBaselineGrowth(baselineStoragePerMonth, baselineDocsPerMonth)'
}

```

Polymer concept — a computed property. This says: "*_baselineGrowth is not a value I set by hand. It is **derived**. Whenever baselineStoragePerMonth or baselineDocsPerMonth changes, automatically re-run _computeBaselineGrowth() with those two as arguments, and store the result in _baselineGrowth.*"

This is **reactive programming**. You never call _computeBaselineGrowth() yourself. You never write "the data arrived, now refresh the table." The dependency chain does it for you:

```

query finishes
→ baselineStoragePerMonth changes
  → Polymer notices (it is listed as a dependency)
    → _computeBaselineGrowth() re-runs automatically
      → _baselineGrowth updates
        → the <table> bound to it re-renders on screen

```

Understanding this chain is the single most valuable thing to be able to explain.

PART 3 — The chart markup

```

<nuxeo-card class="full-width" heading="Growth vs July 2026 (Baseline)">
  <div class="chart-sub-title">
    Documents and storage added per month, starting from the July 2026 baseline
  </div>
  <div class="chart-box">
    <chart-bar
      labels="[[_monthLabels(baselineStoragePerMonth)]]"
      values="[[_baselineValues(baselineStoragePerMonth, baselineDocsPerMonth)]]"
      series="[[_baselineSeries(baselineStoragePerMonth)]]"
      options="[[_overallOptions(baselineStoragePerMonth)]]">
    </chart-bar>
  </div>
  ...
</nuxeo-card>

```

<chart-bar> is a wrapper around **Chart.js**. It takes four inputs:

Attribute	Meaning	Example value
labels	the x-axis text	['Jul 2026', 'Aug 2026', 'Sep 2026']
values	the bar heights	[[500, 750, 400], [1000, 1500, 750]]
series	the legend names	['Storage added in (MB)', 'Number of documents created']

options Chart.js config (axes, colours, legend) an object

Each is a **computed binding** — a helper method is called and its return value is handed to the chart.

Notice `options=["[_overallOptions(baselineStoragePerMonth)]"]` — we **reuse the Overall chart's options function**. That is why the baseline chart looks identical to the Overall chart: same axes, same colours, same legend. Only the data differs.

A hard-won rule. Polymer bindings **cannot nest two function calls**. `[[_a(_b(x))]]` will silently blank the entire page. If you need two steps, do them inside one helper, or add a second helper. This bit us before — worth mentioning to your lead as a real gotcha.

PART 4 — The helper functions, line by line

4.1 `_baselineValues` — turning raw data into bar heights

```
_baselineValues: function(storage, docs) {  
  if (!storage || !storage.length) return [[]];  
  var MB = 1048576;  
  var storageVals = storage.map(function(entry) {  
    var v = (entry.value || 0) / MB;  
    return Math.round(v * 100) / 100;  
  });  
  var docMap = {};  
  (docs || []).forEach(function(d) { docMap[d.key] = d.value || 0; });  
  var docVals = storage.map(function(entry) { return docMap[entry.key] || 0; });  
  return [storageVals, docVals];  
},
```

`function(storage, docs)` Receives the two arrays from the queries. Polymer passes them automatically because they were named in the binding.

`if (!storage || !storage.length) return [[]];` A **guard clause**. When the page first loads, the query has not returned yet, so `storage` is empty. Without this line the code below would crash on `undefined`. Returning `[[]]` (an array containing one empty array) gives the chart a valid but empty shape to draw. **Every helper must tolerate being called before the data exists** — Polymer calls them eagerly.

`var MB = 1048576;` Bytes in one megabyte (1024×1024).

`var storageVals = storage.map(function(entry) {...}) .map()` is a JavaScript array method: "run this function on every element and give me back a new array of the results." Input `[{key,value},{key,value}]` → output `[500, 750]`. It does not modify the original.

`var v = (entry.value || 0) / MB;` Convert bytes to MB. `(entry.value || 0)` means "use `entry.value`, but if it is `undefined/null`, use `0` instead" — a null-safety idiom.

Why MB and not GB? Storage in bytes is a number like 524,288,000 while document counts are ~1,000. On a shared axis the tiny bar would be invisible next to the enormous one. Dividing by 1 MB brings storage down to ~500 — the same order of magnitude as the document count, so **both bars are visible together**. This is a deliberate design decision, and a good one to be able to justify.

`return Math.round(v * 100) / 100;` Round to 2 decimal places. (Multiply by 100, round to a whole number, divide back.)

The alignment problem — the subtle part

```
var docMap = {};  
(docs || []).forEach(function(d) { docMap[d.key] = d.value || 0; });  
var docVals = storage.map(function(entry) { return docMap[entry.key] || 0; });
```

Two separate queries returned two separate arrays. **They might not have the same months**. If no files were uploaded in August, the `storage` array may have an August bucket while the `docs` array does not — or vice versa. If we naively did `docs.map(d => d.value)` the two bar series could be misaligned, and August's document count would be drawn above September's storage bar. Silently wrong data.

The fix, in three steps:

1. `var docMap = {};` — create an empty lookup object.
2. `forEach(...)` — fill it: `{ '2026-07': 1000, '2026-08': 1500 }`. Now we can look up any month's document

count instantly by its key.

3. `storage.map(entry => docMap[entry.key] || 0)` — walk the **storage** array (our reference order) and, for each of *its* months, pull the matching document count out of the map. If a month is missing from the docs data, `|| 0` supplies a zero.

The result: `docVals` is guaranteed to be the same length and the same order as `storageVals`. **The storage array is the single source of truth for the month order, and everything else is aligned to it.**

`return [storageVals, docVals];` An array of two arrays — Chart.js reads this as two data series (two bars per month).

4.2 _baselineSeries — the legend text

```
_baselineSeries: function(storage) {  
  return ['Storage added in (MB)', 'Number of documents created'];  
},
```

Returns the two legend labels, in the same order as the two arrays returned by `_baselineValues`. Order matters: `series[0]` labels `values[0]`.

It takes `storage` as an argument even though it never uses it. That is intentional: a Polymer computed binding only re-runs when one of its listed arguments changes. Passing `storage` makes the legend refresh in step with the data.

4.3 _computeBaselineGrowth — the percentage table

```
_computeBaselineGrowth: function(storage, docs) {  
  if (!storage || !storage.length) return [];  
  var self = this;  
  var docMap = {};  
  (docs || []).forEach(function(d) { docMap[d.key] = d.value || 0; });  
  
  var baseDocs = docMap[storage[0].key] || 0;  
  var baseStorage = storage[0].value || 0;  
  
  function pct(current, base) {  
    if (!base) { return current > 0 ? 'n/a' : '0%'; }  
    var change = ((current - base) / base) * 100;  
    var rounded = Math.round(change * 10) / 10;  
    return (rounded > 0 ? '+' : '') + rounded + '%';  
  }  
  
  return storage.map(function(entry, i) {  
    var d = docMap[entry.key] || 0;  
    var st = entry.value || 0;  
    return {  
      month: self._toMonthYear(entry.key),  
      docs: self._withCommas(d),  
      storage: self._formatStorage(st),  
      docsPct: i === 0 ? 'baseline' : pct(d, baseDocs),  
      storagePct: i === 0 ? 'baseline' : pct(st, baseStorage)  
    };  
  });  
},
```

`var self = this;` A classic JavaScript idiom. Inside the `function(entry, i)` callback below, the keyword `this` no longer refers to our component (it gets rebound). By saving it as `self` beforehand, we can still reach our other methods as `self._withCommas(...)`. Without this line you get "undefined is not a function".

`var baseDocs = docMap[storage[0].key] || 0;` **This line defines the baseline.** `storage[0]` is the *first* bucket the query returned — and because the query starts at July 2026, the first bucket **is** July. `storage[0].key` is '2026-07', so `baseDocs` is July's document count. Every percentage below is measured against this number.

Note the elegance: we never hard-code "July" here. We just say "the first bucket is the baseline." Change `baselineStart` to any month and this code needs no edit.

The pct function

```
if (!base) { return current > 0 ? 'n/a' : '0%'; }
```

Guard against **division by zero**. If July had 0 documents, we cannot compute a percentage increase from zero, so we return 'n/a' rather than Infinity or NaN.

```
var change = ((current - base) / base) * 100;
```

The standard percentage-change formula: $(new - old) \div old \times 100$.

- 1500 vs 1000 $\rightarrow (500 / 1000) \times 100 = +50\%$
- 750 vs 1000 $\rightarrow (-250 / 1000) \times 100 = -25\%$
- 0 vs 1000 $\rightarrow (-1000 / 1000) \times 100 = -100\%$ (an empty month is genuinely a 100% drop)

```
var rounded = Math.round(change * 10) / 10;
return (rounded > 0 ? '+' : '-') + rounded + '%';
```

Round to 1 decimal, and prepend a + for positive values (JavaScript adds the - for negatives automatically).

The .map() that builds the rows

```
return storage.map(function(entry, i) { ... });
```

.map() gets a second argument, *i* — the **index** (0, 1, 2...). We use it to detect the first row.

```
docsPct: i === 0 ? 'baseline' : pct(d, baseDocs),
```

A **ternary operator** (condition ? valueIfTrue : valueIfFalse). For row 0 (July itself) we print the word 'baseline' — because comparing July to July would always be a meaningless "+0%". Every other row gets a real percentage.

Each iteration returns an object shaped exactly like a table row:

```
{ month: 'Aug 2026', docs: '1,500', storage: '750 MB',
  docsPct: '+50%', storagePct: '+50%' }
```

The helpers `_toMonthYear`, `_withCommas` and `_formatStorage` are pre-existing formatting utilities shared with the rest of the dashboard — we reuse rather than duplicate them.

4.4 _hasBaseline and _deltaClass

```
_hasBaseline: function(rows) {
  return !! (rows && rows.length);
},
```

Returns true only if there are rows. `!!` is a shorthand that converts any value to a real boolean. Used by `dom-if` to hide the table entirely while data is loading.

```
_deltaClass: function(pctText) {
  if (!pctText || pctText === 'baseline' || pctText === 'n/a') return 'delta-cell delta-flat';
  if (pctText.charAt(0) === '+') return 'delta-cell delta-up';
  if (pctText.charAt(0) === '-') return 'delta-cell delta-down';
  return 'delta-cell delta-flat';
},
```

Returns a **CSS class name** based on the text. It simply inspects the first character: + $\rightarrow$ green, - $\rightarrow$ red, anything else $\rightarrow$ grey. The component decides the *meaning*; the stylesheet decides the *colour*.

PART 5 — The table markup (and two more Polymer concepts)

```
<template is="dom-if" if="[_hasBaseline(_baselineGrowth)]">
  <div class="baseline-table-wrap">
    <div class="chart-sub-title">Change vs July 2026</div>
    <table class="baseline-table">
      <thead>
        <tr>
          <th>Month</th><th>Documents</th><th>vs Jul</th>
          <th>Storage</th><th>vs Jul</th>
        </tr>
      </thead>
      <tbody>
        <template is="dom-repeat" items="[_baselineGrowth]">
          <tr>
            <td>[[item.month]]</td>
            <td class="num-cell">[[item.docs]]</td>
            <td class$="[_deltaClass(item.docsPct)]">[[item.docsPct]]</td>
            <td class="num-cell">[[item.storage]]</td>
          </tr>
        </template>
      </tbody>
    </table>
  </div>
</template>
```

```

        <td class$="[[_deltaClass(item.storagePct)]]">[[item.storagePct]]</td>
      </tr>
    </template>
  </tbody>
</table>
</div>
</template>

```

```
<template is="dom-if" if="...">
```

Polymer concept — dom-if. Conditional rendering. If the expression is falsy, the entire block is **not created in the DOM at all** (it is not merely hidden with CSS — it does not exist). Here it keeps the empty table off the screen until the query returns.

```
<template is="dom-repeat" items="[[_baselineGrowth]]">
```

Polymer concept — dom-repeat. A loop. It stamps out one copy of the enclosed markup for **every element** in items. Inside the loop, Polymer gives you a magic variable called **item** — the current element. So `[[item.month]]` reads the month field of the current row object.

Five rows in `_baselineGrowth` → five `<tr>` elements on screen. Add a sixth month to the data and a sixth row appears with no code change.

Requirement: the bound property *must* be a real Array with a `[]` default, which is exactly why we declared type: Array, value: `function(){ return []; }`. Bind dom-repeat to a non-array and you get the console error *"dom-repeat expected array for items"*.

```
class$="[[_deltaClass(item.docsPct)]]" — note the dollar sign
```

Polymer concept — attribute binding (\$=). There are two kinds of binding:

- `class="..."` sets a JavaScript **property** on the element.
- `class$="..."` sets the actual HTML **attribute**.

For native HTML attributes like class, href, colspan and style, you **must** use `$=` or the value will not appear in the markup and your CSS will not match. This is a very common Polymer beginner bug. Here it is what lets the cell turn green or red.

PART 6 — The styles

```

.baseline-table {
  width: 100%;
  border-collapse: collapse; /* single shared borders, not doubled */
  font-size: 13px;
  font-family: var(--nuxeo-app-font, 'Roboto', sans-serif);
}
.baseline-table thead th {
  text-align: left;
  font-weight: 600;
  font-size: 11px;
  letter-spacing: 0.4px;
  text-transform: uppercase; /* headers read as labels, not data */
  color: #8a938c; /* muted grey – recedes behind the numbers */
  padding: 8px 12px;
  border-bottom: 1px solid #e4e9e4;
  white-space: nowrap; /* never wrap a header mid-word */
}
.baseline-table tbody td {
  padding: 9px 12px;
  border-bottom: 1px solid #f2f5f2;
  color: #33413a;
}
.baseline-table tbody tr:last-child td { border-bottom: none; }
.baseline-table tbody tr:hover { background: #f7faf7; }

.baseline-table .num-cell {
  font-variant-numeric: tabular-nums; /* digits get equal width */
  white-space: nowrap;
}
.delta-cell {
  font-weight: 600;
  font-variant-numeric: tabular-nums;
}

```



```

    white-space: nowrap;
  }
  .delta-up { color: #2e7d32; } /* Fidelity green - growth */
  .delta-down { color: #c0392b; } /* red - decline */
  .delta-flat { color: #9aa39c; font-weight: 500; } /* grey - baseline / n/a */

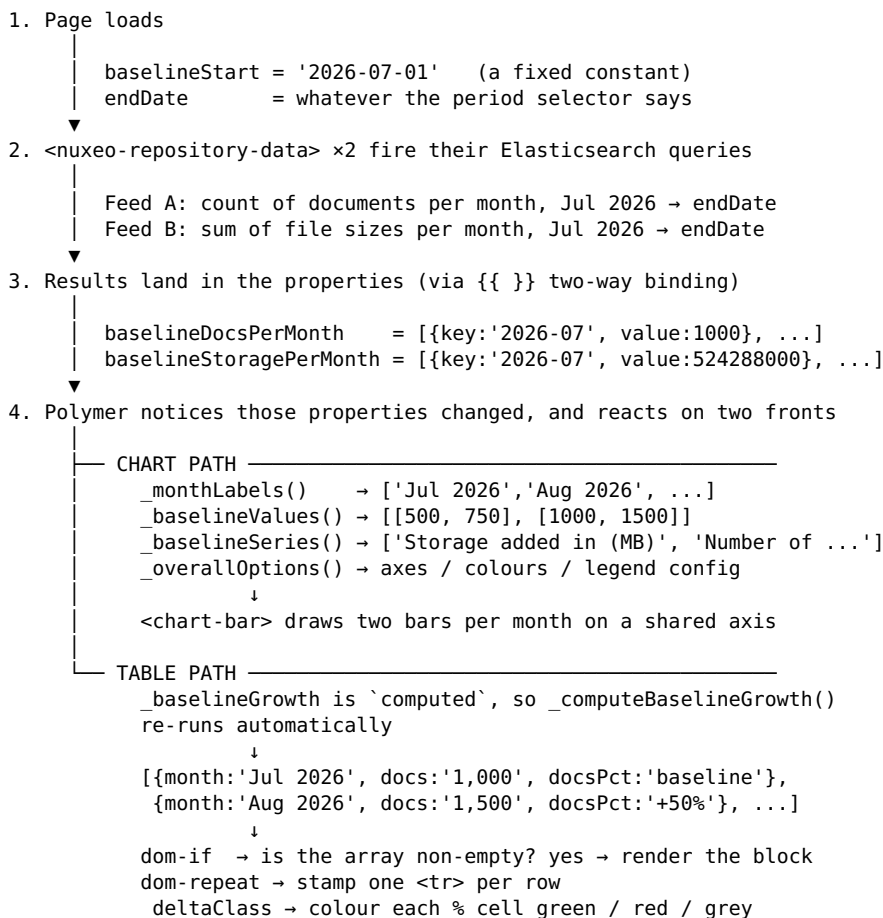
```

Two choices worth being able to defend:

font-variant-numeric: tabular-nums — in most fonts a 1 is narrower than a 8, so columns of figures look ragged. This forces every digit to the same width so numbers line up vertically in a column. It is what makes a data table look professional rather than homemade.

var(--nuxeo-app-font, 'Roboto', sans-serif) — a CSS custom property with a fallback. Use Nuxeo's configured app font if it exists; otherwise Roboto; otherwise any sans-serif. The component inherits the host application's typography instead of imposing its own.

PART 7 — The complete program flow



Nothing in this flow is triggered manually. There is no "refresh" button, no event listener, no `updateTable()` call. The user changes the end date → the query re-runs → the properties change → every binding that depends on them recalculates. That is Polymer's reactive model, and it is the key idea to convey.

PART 8 — A full worked example (trace this end to end)

Setup: the user picks an end date of 30 September 2026.

Step 1 — the queries run. start-date = 2026-07-01 (fixed), end-date = 2026-09-30T23:59:59.

Step 2 — Elasticsearch returns:

```

baselineDocsPerMonth = [
  { key: '2026-07', value: 1000 },
  { key: '2026-08', value: 1500 },

```



```

    { key: '2026-09', value: 750 }
  ]

  baselineStoragePerMonth = [
    { key: '2026-07', value: 524288000 }, // 500 MB
    { key: '2026-08', value: 786432000 }, // 750 MB
    { key: '2026-09', value: 419430400 }  // 400 MB
  ]

```

Step 3 — `_baselineValues()` runs.

```

storageVals: 524288000 / 1048576 = 500      → [500, 750, 400]
docMap      : { '2026-07':1000, '2026-08':1500, '2026-09':750 }
docVals     : walk storage's months, look each up → [1000, 1500, 750]
returns     : [[500, 750, 400], [1000, 1500, 750]]

```

The chart draws, per month, a storage bar and a document bar side by side.

Step 4 — `_computeBaselineGrowth()` runs.

```

baseDocs    = docMap['2026-07']      = 1000
baseStorage = storage[0].value        = 524288000

```

```

Row 0 (July)   i === 0 → docsPct = 'baseline', storagePct = 'baseline'

```

```

Row 1 (August)
  docs    : pct(1500, 1000) = ((1500-1000)/1000)*100 = +50 → '+50%'
  storage : pct(786432000, 524288000) = +50      → '+50%'

```

```

Row 2 (September)
  docs    : pct(750, 1000) = ((750-1000)/1000)*100 = -25 → '-25%'
  storage : pct(419430400, 524288000) = -20      → '-20%'

```

Result:

```

[
  { month:'Jul 2026', docs:'1,000', storage:'500 MB', docsPct:'baseline', storagePct:'baseline' },
  { month:'Aug 2026', docs:'1,500', storage:'750 MB', docsPct:'+50%', storagePct:'+50%' },
  { month:'Sep 2026', docs:'750', storage:'400 MB', docsPct:'-25%', storagePct:'-20%' }
]

```

Step 5 — the table renders. `dom-if` sees 3 rows → renders. `dom-repeat` stamps 3 `<tr>`s. `_deltaClass('+50%')` returns delta-up → green. `_deltaClass('-25%')` returns delta-down → red.

On screen:

Month	Documents	vs Jul	Storage	vs Jul
Jul 2026	1,000	<i>baseline</i>	500 MB	<i>baseline</i>
Aug 2026	1,500	+50% (green)	750 MB	+50% (green)
Sep 2026	750	-25% (red)	400 MB	-20% (red)

Step 6 — the user changes the end date to 31 October. The end-date binding changes → both queries re-run → the properties change → `_baselineValues` and `_computeBaselineGrowth` re-run automatically → a fourth bar and a fourth table row appear. **July is still the baseline, because `baselineStart` never moved.** No code ran to make that happen; the bindings did it.

PART 9 — Talking points for your lead

If you can say these five things you understand the feature:

1. **"The baseline is enforced at the query layer, not in the chart."** `start-date` is bound to a constant (`baselineStart`) instead of the period selector. The chart is dumb — it just draws whatever months it is given. That is why nothing the user does in the dropdown can move the baseline.
2. **"The two feeds are aligned by key, not by index."** Storage and document counts come from two independent queries that may not return identical month buckets. Building a `docMap` and walking the storage array keeps the two bar series in lockstep. Zipping them by position would silently misalign the data.
3. **"Storage is converted to MB so both series share one axis."** Raw bytes (~500,000,000) would

dwarf document counts (~1,000) and flatten the count bars to invisibility. MB puts both on the same order of magnitude.

4. **"The percentage table is a computed property, so it is self-updating."** `_baselineGrowth` declares its dependencies; Polymer re-runs the calculation whenever they change. There is no imperative refresh code anywhere.
5. **"Row 0 is the baseline by definition, not by date-matching."** Because the query starts at the baseline month, `storage[0]` is the baseline bucket. Changing `baselineStart` re-baselines the entire feature — one line, no other edits.

And one honest limitation worth volunteering: the baseline chart deliberately ignores the period selector's *start* date, but it does still follow the *end* date. So changing the period changes how far the chart runs, not where it begins. That is the intended behaviour, but it is the sort of thing a lead will probe — better to raise it yourself than be caught by it.

Appendix — Quick reference

Concept	Syntax	Meaning
One-way binding	<code>[[foo]]</code>	display a value (parent → child)
Two-way binding	<code>{{foo}}</code>	child can write back (needed for query results)
Computed binding	<code>[[_fn(a, b)]]</code>	call a method, use its return value
Computed property	<code>computed: '_fn(a, b)'</code>	auto-recalculate when a or b change
Conditional	<code><template is="dom-if" if="..."></code>	render only when true
Loop	<code><template is="dom-repeat" items="..."></code>	one copy per array element; <code>item</code> = current
Attribute binding	<code>class\$="[[...]]"</code>	required for class, style, href
Array/Object default value:	<code>function() { return []; }</code>	must be a function, never a literal
Gotcha	Rule	
Nested bindings	<code>[[_a(_b(x))]]</code>	blanks the page — never nest two calls
dom-repeat needs a real array	declare type: <code>Array</code> with a <code>[]</code> function default	
CSS not applying?	you probably wrote <code>class=</code> where you needed <code>class\$=</code>	
Helper crashes on load	add a guard: <code>`if (!data</code>	
Two data sources	align by key , never by array position	